

Now, let us check whether the undirected multigraph *Mgraph_7* contains a Hamiltonian circuit. To do that, add the following line of code at the end of the program called *'mgraph7.sagews'*.

```
MG.hamiltonian_cycle( )
```

Upon execution of the modified program, we get an error message displayed on the screen. The issue again stems from the loops and multiple edges present in our graph *Mgraph_7*. The *hamiltonian_cycle()* method has problems handling loops and multiple edges. The fix is the same as before — create a new graph without loops and multiple edges from the original graph, and then check for a Hamiltonian cycle in this new graph. This time the answer will be accurate. If an undirected simple graph has a Hamiltonian cycle, then the same Hamiltonian cycle will be present even if we add multiple edges and loops. In general, if a graph has a Hamiltonian cycle, it will continue to have that cycle even if we add more edges. However, the same is not true about Eulerian cycles in a graph. Please think about some examples to verify that the statement I made is indeed true.

The SageMath methods we discussed just now clearly indicate that we must be careful when dealing with multigraphs. Methods that work for simple graphs may not work for multigraphs. However, with SageMath being free and open source software, you can edit the source code to create methods that suit your needs once you gain sufficient mastery.

Now, it is time for us to familiarise ourselves with one more type of undirected simple graph: the tree. A *tree* is a graph without any cycles. Trees can also be used to represent many real-world scenarios and problems. For example, the knockout stage of any sports tournament can be represented using a tree. The program *'tree8.sagews'* shown below generates a tree called the *Tree_8*.

```
T = Graph( )
teams = ['S1_T1_South Africa', 'S1_T2_Afghanistan', 'S2_T1_India', 'S2_T2_England', 'F_T1_South Africa', 'F_T2_India']
T.add_vertices(teams)
T.add_edges([( 'S1_T1_South Africa', 'F_T1_South Africa'),
              ('S1_T2_Afghanistan', 'F_T1_South Africa'),
              ('S2_T1_India', 'F_T2_India'),
              ('S2_T2_England', 'F_T2_India'),
              ('F_T1_South Africa', 'F_T2_India')])
T.plot(vertex_size=600).show( )
```

The code does not need any explanation, as it is similar to the other undirected simple graphs generated earlier in this series. The only difference is that this graph does not contain any cycles. In the last line of the code, we use the *plot()* method to provide the option *vertex_size* so that larger

vertices are displayed by the *show()* method. Upon execution of the program *'tree8.sagews'*, we get the graph called *tree_8* shown in Figure 1. Notice that this is the knockout stage (semifinals and final) of the recently concluded 2024 ICC Men's T20 World Cup. To view a tree with a larger number of vertices, see the Wikipedia page of the 2024 Wimbledon tennis championship. A family tree is another example where a tree can be used to capture real-world information.

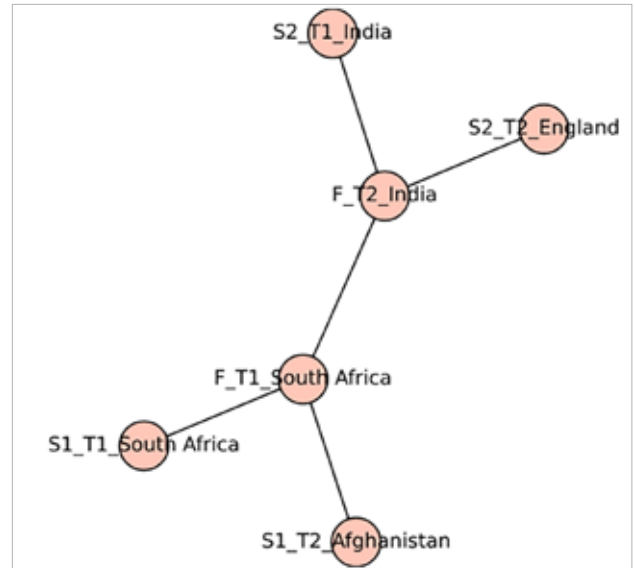


Figure 1: The tree *Tree_8* generated using SageMath

Before we move on from undirected graphs (simple or multigraphs), we need to familiarise ourselves with two more graph-theoretic terms: connected and disconnected graphs. So far, we have only encountered connected graphs in this series. A *connected undirected graph* is a graph in which there is a path between every pair of vertices. Use the method *all_paths()* offered by SageMath to verify the existence of at least one path between any pair of vertices in all the graphs we have discussed so far in this series. On the other hand, a *disconnected graph* is a graph that is not connected. In such a graph, there is at least one pair of vertices that has no path connecting them. The program *'disgraph9.sagews'* shown below generates a graph called *Disgraph_9*, which shows the rail connectivity between several cities/towns in India.

```
G = Graph( )
G.add_vertices(['Mumbai', 'Delhi', 'Chennai', 'Kolkata', 'Port Blair', 'Kavaratti'])
G.add_edges([( 'Mumbai', 'Delhi'),
              ('Delhi', 'Kolkata'),
              ('Kolkata', 'Chennai'),
              ('Chennai', 'Mumbai')])
G.show( )
```